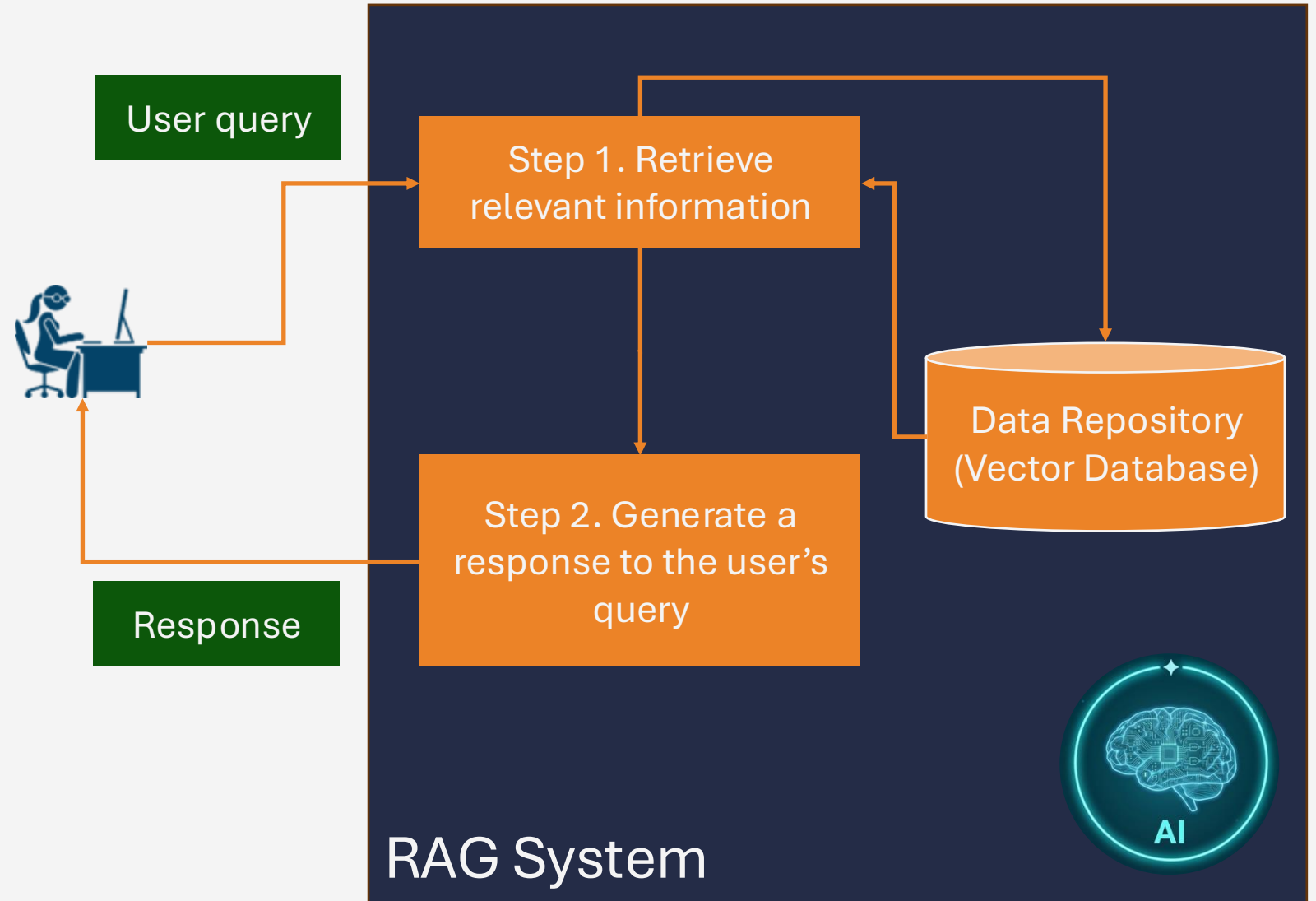


RAG Search Platform Tests

What is RAG?

- AI software components utilizing dedicated internal information when responding to natural-language user queries
- Requirements
 - Data Repository
 - AI logical components to
 - Search and retrieve relevant context
 - Generate a response





RAG Search Research Objectives

1. Test turnkey RAG solutions from AWS and GCP along dimensions of ***ease of deployment*** and ***quality of responses***
2. Provide ***code examples*** helping readers jump start their RAG initiatives
3. Explore techniques for ***improving RAG responses***
4. Provide simple RAG examples helping builders ***develop intuition*** into how these tools work and how they might be improved



Findings

- Construction

- Out-of-the-box RAG solutions are viable and could be optimal in most applications
- Some assembly is required even with out-of-the-box products
- There are many data and technological knobs that can be adjusted even with pre-packaged solutions
- Best results come from feeding LLMs relevant content and direct questions meaning that, in some cases, separating the retrieve (search) and generate (answer) functionalities is beneficial
- Step separation can allow more control over the response format (e.g. long or short)

- Data

- Performance assessment and user-feedback integration are critical
- Data curation including cleaning and enrichment is critical and can dramatically improve performance
- Ongoing performance and input/output data monitoring are required
- Augmenting pre-built application data adapters to normalize data across multiple sources can improve performance

Test Results



RAG Pipeline Construction

1. Data Enrichment and Ingestion	2. RAG Construction	3. RAG Query
• Enrich documents by adding a type source and title • Load documents into text and metadata files	• Construct vector database/index • Ingest documents	• Query data repositories using both retrieve and retrieve-and-generate functionality



S3

Bedrock Knowledge Base



Google Cloud Storage

Vertex AI Search
(Agent Search)

Notebooks

/aws_bedrock/S10_load_data_s3
/gcp_vertexai/S12_load_data_gcs

/aws_bedrock/
S20_build_bedrock_knowledgebase
/gcp_vertexai/
S20_build_vertexai_search_app

/aws_bedrock/
S30_query_bedrock_knowledgebase
/gcp_vertexai/
S30_query_bedrock_knowledgebase



Example Questions

Query	Human Answer	Baseline RAG - Bedrock	RAG with Filtering	AI Client with source text – Vertex AI
What are the key topics of this article?	The key topics of this article are: "why prioritizing impact matters not just for managers, but also ICs"; ... and "how to overcome common challenges in driving real impact".	I cannot determine the key topics of "this article" because no specific article was provided in your question. The search results contain information about various unrelated topics, ...	The article focuses on how data scientists can maximize their impact at work. The key topics include: the importance of focusing on impact rather than just output or effort; ...	The key topics of this article are: - 1**Why prioritizing impact matters** for individual contributors (ICs) ... - 4 **How to overcome common challenges** in driving real impact ...

It's not surprisingly RAG systems without guidance struggled with this question. To see all questions and test results, click the [RAG Search Test Questions](#) link above.

Test results summary

- These charts show RAG search and answer-generation accuracy under various RAG configurations
- In general, best performance is observed when AI clients are prompted with relevant source content
- This is probably not a fair test since we don't need RAG tools if we know ex-ante which documents are relevant
- However, it does tell us that we should help RAG systems by employ all mechanisms to find relevant content

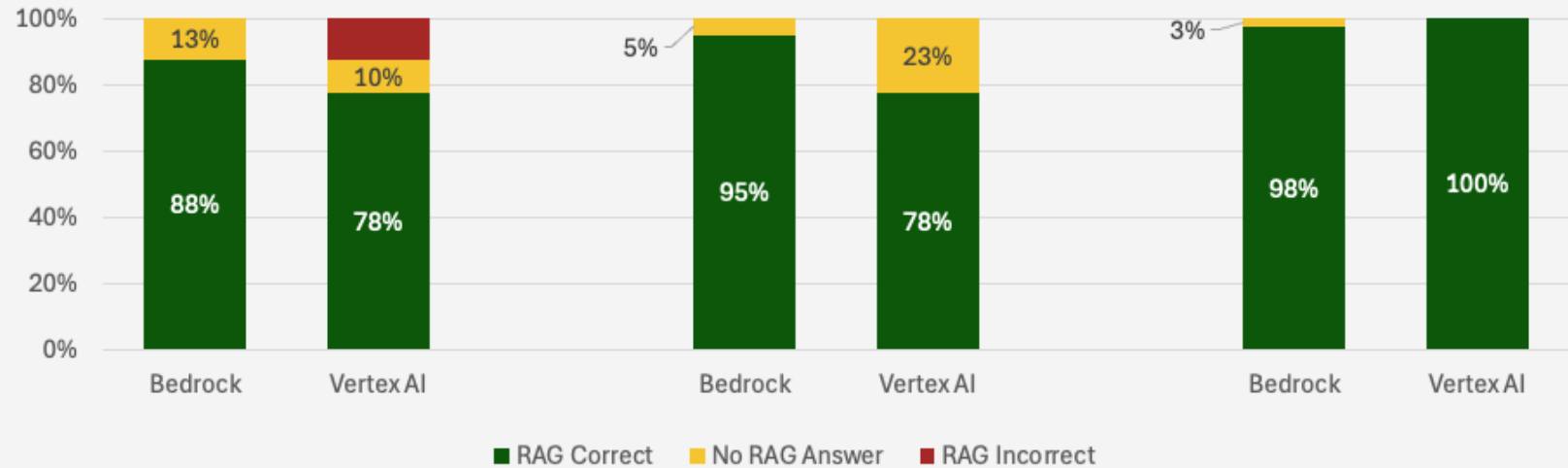


1. Baseline Search and Generate

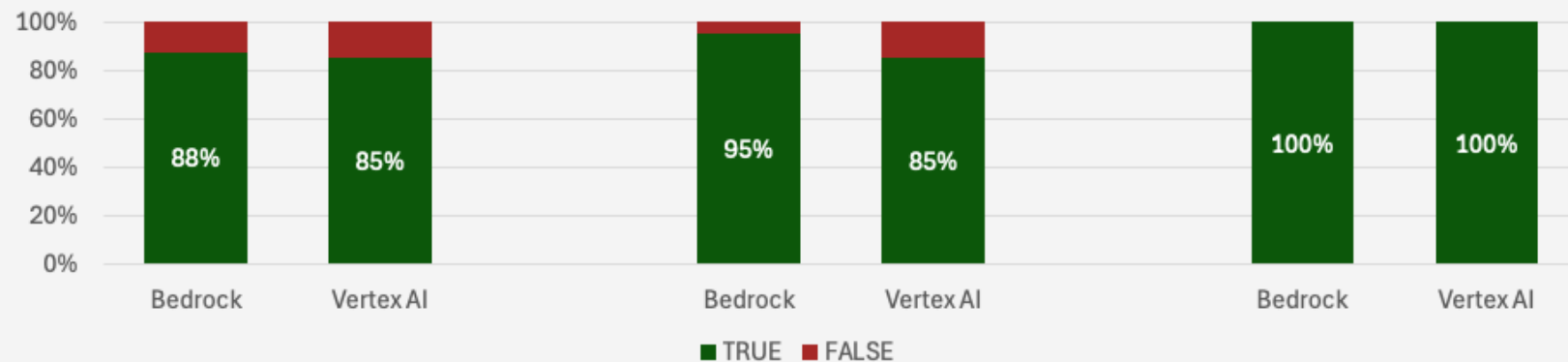
2. Search and Generate with Document Filter

3. AI Client with Source Text

RAG Retrieve and Generate Accuracy



Search Found Correct Source





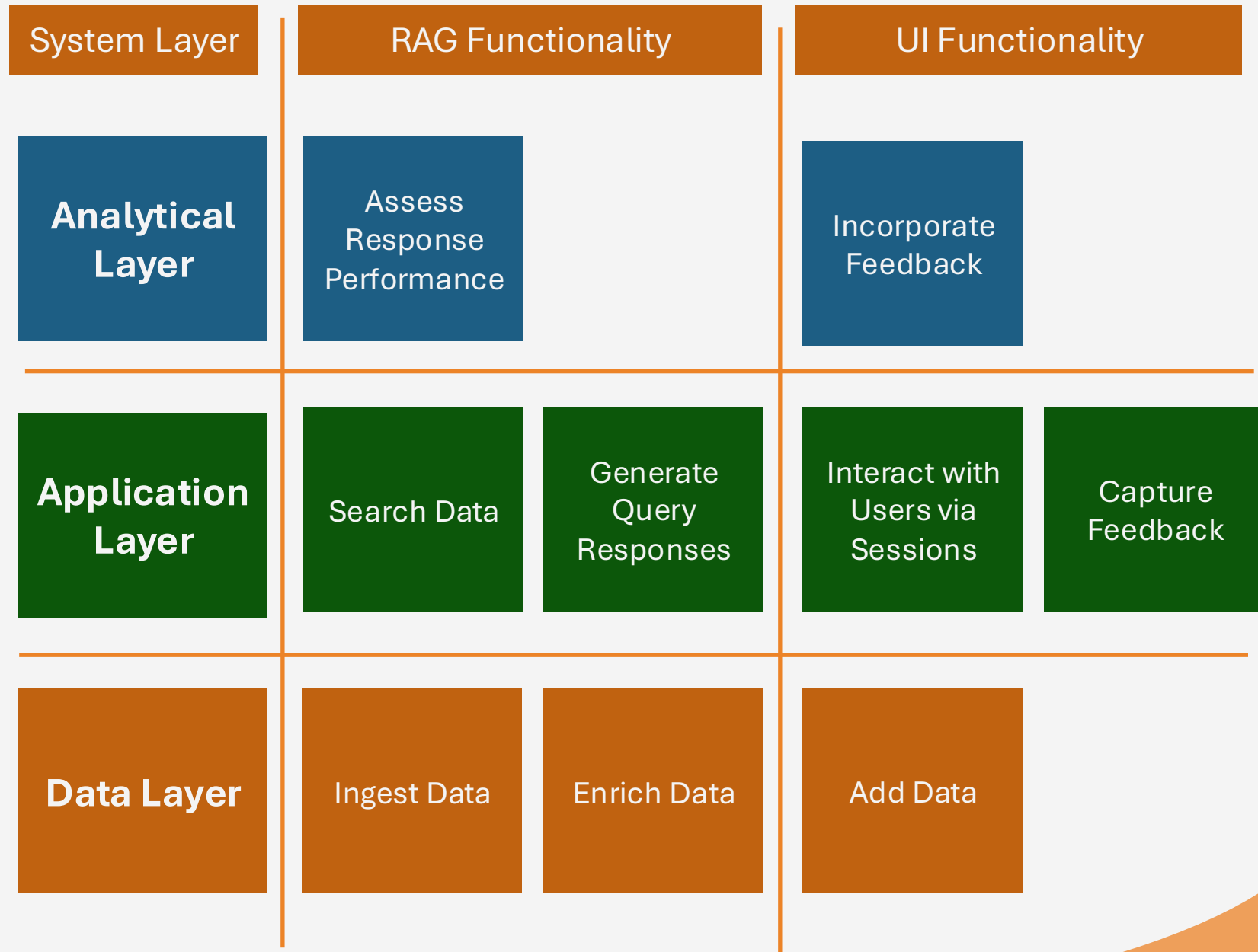
RAG Testing Results

	Description	Results and Comments
1. Baseline Search and Generate	Out-of-the-box retrieve and generate with minimal data curation	• Fairly strong baseline results with 78 - 88% accuracy with Bedrock somewhat better • Given the vagueness of some questions, some level of no response is expected • Vertex AI hallucinations are troubling but can possibly be controlled
2. Search and Generate with Document Filter	Out-of-the-box retrieve and generate with minimal data curation with the addition of filtering to only text from the document source	• Slight improvement in Bedrock performance • Elimination of Vertex AI hallucinations but no improvement in accuracy • Filtering RAG search doesn't seem to result in more extensive and relevant results – rather it appears to narrow the baseline results
3. AI Client with Source Text	AI client prompted with the source text and instructions to provide short answer to query	• Near 100% accuracy • Potentially more control over the output format (e.g. prompting for a long or short) response • Not always realistic to know which source documents are relevant, but, if possible, can be powerful

RAG Construction

Key Architectural Elements

- RAG
 - A search and generate tool providing responses to natural language queries
 - Key components include a
 - **Data Layer** to ingest and enrich data
 - **Application Layer** to search unstructured data and return responses
 - **Analytical Layer** to measure the quality of outputs
- User Interface
 - The UI layer can be used to capture user feedback and additional data improving quality





Out-of-the-box RAG solutions tested in this

Feature	Amazon Bedrock Knowledge Bases	GCP Vertex AI Search (Agent Builder)
Best For	Organizations already deep in the AWS ecosystem who want granular control over the vector database and retrieval settings.	Teams wanting the fastest time-to-market with a "Google-quality" search interface and minimal configuration.
Data Sources	Cloud Native: Amazon S3. SaaS Connectors: SharePoint, Salesforce, Confluence, OneDrive. Web: URL Web Crawler.	Cloud Native: Cloud Storage, BigQuery. Ecosystem: Native Google Workspace (Drive, Gmail). SaaS: Over 100+ connectors (Jira, Slack, Zendesk, etc.). Web: Advanced Google Search indexing.
Vector DB	Flexible. Uses Amazon OpenSearch Serverless by default, but also supports Pinecone, Redis Cloud, MongoDB Atlas, and Aurora.	Abstracted. Built on Vertex AI Vector Search, but the underlying database is managed entirely by the service (you don't see the index directly).
Embeddings	Choice of Titan Embeddings (v1/v2) or Cohere Embeddings .	Uses Google's proprietary Text-Embedding models (e.g., Gecko), optimized specifically for the Search app.
Ease of Setup	Moderate. Streamlined via the console, but requires setting up IAM roles, S3 buckets, and specific vector index permissions.	Very High. "Click-to-sync" functionality. It feels like setting up a consumer-grade search engine rather than a technical pipeline.
Maintenance	Low. Serverless handling of data syncing, though you may still need to monitor the specific Vector DB performance/costs.	Near Zero. It is a fully managed "Search App" experience. Google handles the chunking, indexing, and tuning updates.
Pricing	Modular. You pay for Bedrock API calls + Vector DB storage (OCUs) + S3 storage + Embedding token costs.	Unit-based. Generally based on a flat fee per 1,000 queries + data storage per GiB. Can be more predictable, but often higher "entry" price.



Amazon Bedrock Knowledge Bases

Programmatic Build

Load Source Data	Construct Knowledge Base	Query Data
1. Enrich documents by adding metadata 2. Load test documents into S3	1. Establish security roles 1. Create OpenSearch Serverless Security Policies 2. Create IAM Role for Bedrock 2. Set up Knowledge Base 1. Configuration and Initialization 2. Create vector index 3. Create Knowledge Base using vector index which automatically creates a vector index 3. Ingest source data 1. Add S3 documents to Knowledge Base	1. Retrieve documents with pure search 2. Answer queries using AI RAG 3. Answer queries with filters



Vertex AI Search Programmatic Build

Load Source Data

1. Parse and load test documents into BigQuery

Build Vertex AI Search App

1. Enable APIs and establish security roles
2. Create a Vertex AI Datastore
3. Update the datastore's schema
4. Ingest documents into the datastore
5. Create a Vertex AI Search App

Query Data

1. Retrieve and answer queries



Test Data

- Dataset

- [Single-Topic RAG Evaluation Dataset](#)
- This dataset was designed to evaluate the performance of RAG AI querying text documents about a single topic with word counts ranging from a few thousand to a few tens of thousands, such as articles, blogs, and documentation. The sources were intentionally chosen to have been produced within the last few years (from the time of writing in July 2024) and to be relatively niche, to reduce the chance of evaluated LLMs including this information in their training datasets.
- Our use
 - We selected a small test dataset to help establish a sense for how RAG pipelines are responding
 - To simplify, we use only the 40 single-passage questions